

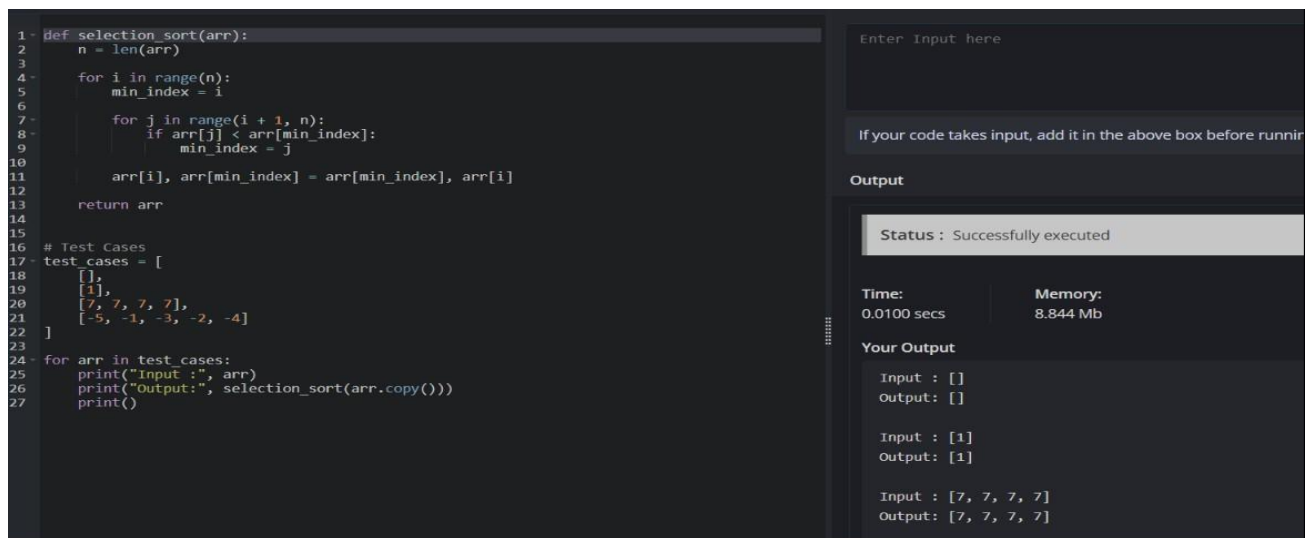
# UNIT – 2 LAB EXERCISE

Name: Jeeva

Reg no: 192421334

1. Write a program to perform the following

- An empty list
- A list with one element
- A list with all identical elements
- A list with negative numbers



```
1 def selection_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         min_index = i
6
7         for j in range(i + 1, n):
8             if arr[j] < arr[min_index]:
9                 min_index = j
10
11         arr[i], arr[min_index] = arr[min_index], arr[i]
12
13     return arr
14
15
16 # Test Cases
17 test_cases = [
18     [],
19     [1],
20     [7, 7, 7, 7],
21     [-5, -1, -3, -2, -4]
22 ]
23
24 for arr in test_cases:
25     print("Input :", arr)
26     print("Output:", selection_sort(arr.copy()))
27     print()
```

Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Successfully executed

|             |          |
|-------------|----------|
| Time:       | Memory:  |
| 0.0100 secs | 8.844 Mb |

Your Output

Input : []  
Output: []

Input : [1]  
Output: [1]

Input : [7, 7, 7, 7]  
Output: [7, 7, 7, 7]

2. Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm's mechanics and effectiveness.

The screenshot shows a Python3 IDE with the following code in the editor:

```
1 def selection_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         min_index = i
6
7         for j in range(i + 1, n):
8             if arr[j] < arr[min_index]:
9                 min_index = j
10
11         arr[i], arr[min_index] = arr[min_index], arr[i]
12
13     return arr
14
15 arr = list(map(int, input("Enter elements: ").split()))
16 print("Sorted Array:", selection_sort(arr))
```

The right sidebar shows the execution output:

- Input: 5 2 9 1 5 6
- Status: Successfully executed
- Time: 0.0100 secs, Memory: 9.288 Mb
- Sample Input: 5 2 9 1 5 6
- Your Output: Enter elements: Sorted Array: [1, 2, 5, 5, 6, 9]

3. Write code to modify bubble\_sort function to stop early if the list becomes sorted before all passes are completed.

The screenshot shows a Python3 IDE with the following code in the editor:

```
1 def bubble_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         swapped = False
6
7         for j in range(0, n - i - 1):
8
9             if arr[j] > arr[j + 1]:
10                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
11                 swapped = True
12
13         if not swapped:
14             break
15
16     return arr
17
18 arr = list(map(int, input("Enter elements: ").split()))
19 print("Sorted Array:", bubble_sort(arr))
```

The right sidebar shows the execution output:

- Input: 64 25 12 22 11
- Status: Successfully executed
- Time: 0.0000 secs, Memory: 9.252 Mb
- Sample Input: 64 25 12 22 11
- Your Output: Enter elements: Sorted Array: [11, 12, 22, 25, 64]

4. Test your optimized function with the following lists:

- Input: [64, 25, 12, 22, 11]
- Expected Output: [11, 12, 22, 25, 64]
- Input: [29, 10, 14, 37, 13]
- Expected Output: [10, 13, 14, 29, 37]
- Input: [3, 5, 2, 1, 4]
- Expected Output: [1, 2, 3, 4, 5]

```

Python3
1 def bubble_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         swapped = False
6
7         for j in range(0, n - i - 1):
8
9             if arr[j] > arr[j + 1]:
10                arr[j], arr[j + 1] = arr[j + 1], arr[j]
11                swapped = True
12
13        if not swapped:
14            break
15
16    return arr
17
18 arr = list(map(int, input("Enter elements: ").split()))
19
20 print("Sorted Array:", bubble_sort(arr))
21

```

64 25 12 22 11

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 9.252 Mb

Sample Input

64 25 12 22 11

Your Output

Enter elements: Sorted Array: [11, 12, 22, 25, 64]

- Write code for Insertion Sort that manages arrays with duplicate elements during the sorting process. Ensure the algorithm's behavior when encountering duplicate values, including whether it preserves the relative order of duplicates and how it affects the overall sorting outcome

```

1 def insertion_sort(arr):
2
3     for i in range(1, len(arr)):
4         key = arr[i]
5         j = i - 1
6
7         while j >= 0 and arr[j] > key:
8             arr[j + 1] = arr[j]
9             j -= 1
10
11        arr[j + 1] = key
12
13    return arr
14
15 arr = list(map(int, input("Enter elements: ").split()))
16
17 print("Sorted Array:", insertion_sort(arr))
18

```

3 1 4 1 5 9 2 6 5 3

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 9.128 Mb

Sample Input

3 1 4 1 5 9 2 6 5 3

Your Output

Enter elements: Sorted Array: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

- Given an array `arr` of positive integers sorted in a strictly increasing order, and an integer `k`. return the `k`th positive integer that is missing from this array.

```

Python3
1 def find_kth_positive(arr, k):
2     missing = 0
3     current = 1
4     i = 0
5
6     while True:
7         if i < len(arr) and arr[i] == current:
8             i += 1
9         else:
10            missing += 1
11            if missing == k:
12                return current
13            current += 1
14
15 arr = list(map(int, input("Enter sorted array: ").split()))
16 k = int(input("Enter k: "))
17
18 print("Kth Missing Positive Number:", find_kth_positive(arr, k))

```

2 3 4 7 11  
5

Output

Status : Successfully executed

Time: 0.0200 secs      Memory: 9.164 Mb

Sample Input

2 3 4 7 11  
5

Your Output

Enter sorted array: Enter k: Kth Missing Positive Number: 9

7. A peak element is an element that is strictly greater than its neighbors. Given a 0indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks. You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array. You must write an algorithm that runs in  $O(\log n)$  time.

```

Python3
1 def find_peak(nums):
2     left = 0
3     right = len(nums) - 1
4
5     while left < right:
6         mid = (left + right) // 2
7
8         if nums[mid] > nums[mid + 1]:
9             right = mid
10        else:
11            left = mid + 1
12
13    return left
14
15
16 nums = list(map(int, input("Enter elements: ").split()))
17 index = find_peak(nums)
18
19 print("Peak Element Index:", index)
20 print("Peak Element:", nums[index])
21

```

1 2 3 1

Output

Status : Successfully executed

Time: 0.0100 secs      Memory: 9.188 Mb

Sample Input

1 2 3 1

Your Output

Enter elements: Peak Element Index: 2  
Peak Element: 3

8. Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or -1 if `needle` is not part of `haystack`.

```
def str_str(haystack, needle):
    n = len(haystack)
    m = len(needle)
    for i in range(n - m + 1):
        if haystack[i:i + m] == needle:
            return i
    return -1

haystack = input("Enter haystack: ")
needle = input("Enter needle: ")
print("First Occurrence Index:", str_str(haystack, needle))
```

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.84 Mb

Sample Input

sadbutsad  
sad

Your Output

Enter haystack: Enter needle: First Occurrence Index: 0

9. Given an array of string words, return all strings in words that is a substring of another word. You can return the answer in any order. A substring is a contiguous sequence of characters within a string

```
def str_str(haystack, needle):
    n = len(haystack)
    m = len(needle)
    for i in range(n - m + 1):
        if haystack[i:i + m] == needle:
            return i
    return -1

haystack = input("Enter haystack: ")
needle = input("Enter needle: ")
print("First Occurrence Index:", str_str(haystack, needle))
```

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.84 Mb

Sample Input

sadbutsad  
sad

Your Output

Enter haystack: Enter needle: First Occurrence Index: 0

10. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

```

1 import math
2
3 def closest_pair(points):
4     min_dist = float('inf')
5     pair = ()
6
7     n = len(points)
8
9     for i in range(n):
10        for j in range(i + 1, n):
11
12            dist = math.sqrt((points[i][0] - points[j][0]) ** 2 +
13                             (points[i][1] - points[j][1]) ** 2)
14
15            if dist < min_dist:
16                min_dist = dist
17                pair = (points[i], points[j])
18
19     return pair, min_dist
20
21 points = [(1, 2), (4, 5), (7, 8), (3, 1)]
22
23 pair, distance = closest_pair(points)
24
25 print("Closest Pair:", pair)
26 print("Minimum Distance:", distance)

```

Enter Input here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.244 Mb

Your Output

Closest Pair: ((1, 2), (3, 1))  
Minimum Distance: 2.23606797749979

11. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7 (6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the sameline?

```

1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2)
5
6
7 def closest_pair(points):
8     n = len(points)
9     min_dist = float('inf')
10    pair = None
11
12    for i in range(n):
13        for j in range(i + 1, n):
14            d = distance(points[i], points[j])
15
16            if d < min_dist:
17                min_dist = d
18                pair = (points[i], points[j])
19
20    return pair, min_dist
21
22
23 n = int(input("Enter number of points: "))
24 points = []
25
26 for i in range(n):
27     x, y = map(int, input(f"Enter point {i+1} (x y): ").split())
28     points.append((x, y))
29
30 pair, dist = closest_pair(points)
31
32 print("Closest Pair:", pair)
33 print("Minimum Distance:", dist)
34 print("Time complexity: O(n^2)")

```

1 4  
2 7  
3 5  
3 6

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.42 Mb

Sample Input

4  
1 4  
2 7  
3 5  
3 6

Your Output

Enter number of points: Enter point 1 (x y): Enter point 2 (x y): Enter point 3 (x y):  
Closest Pair: ((2, 7), (3, 6))  
Minimum Distance: 1.4142135623730951  
Time Complexity: O(n^2)

12. Write a program that finds the convex hull of a set of 2D points using the brute force approach.

```
1 def orientation(p, q, r):
2     return (q[1] - p[1]) * (r[0] - q[0]) - (q[0] - p[0]) * (r[1] - q[1])
3
4
5 def convex_hull(points):
6     n = len(points)
7
8     if n < 3:
9         return points
10
11     hull = []
12
13     left = 0
14     for i in range(1, n):
15         if points[i][0] < points[left][0]:
16             left = i
17
18     p = left
19
20     while True:
21         hull.append(points[p])
22
23         q = (p + 1) % n
24
25         for i in range(n):
26             if orientation(points[p], points[i], points[q]) < 0:
27                 q = i
28
29         p = q
30
31         if p == left:
32             break
33
34     return hull
35
36
37 points = [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]
```

Output

Status : Successfully executed

Time: 0.0100 secs      Memory: 9.1 Mb

Sample Input

Your Output

Convex Hull: [(0, 0), (8, 1), (4, 6)]

13. You are given a list of cities represented by their coordinates. Develop a program that utilizes exhaustive search to solve the TSP.

```
1 from itertools import permutations
2 import math
3
4
5 def distance(c1, c2):
6     return math.sqrt((c1[0]-c2[0])**2 + (c1[1]-c2[1])**2)
7
8
9 def tsp(cities):
10     start = cities[0]
11     others = cities[1:]
12
13     min_distance = float('inf')
14     best_path = None
15
16     for perm in permutations(others):
17         path = [start] + list(perm) + [start]
18
19         total = 0
20
21         for i in range(len(path)-1):
22             total += distance(path[i], path[i+1])
23
24         if total < min_distance:
25             min_distance = total
26             best_path = path
27
28     return min_distance, best_path
29
30
31 cities = [(1,2), (4,5), (7,1), (3,6)]
32 dist, path = tsp(cities)
```

Output

Status : Successfully executed

Time: 0.0100 secs      Memory: 9.28 Mb

Sample Input

Your Output

Shortest Distance: 16.969112047670894  
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]

14. You are given a cost matrix where each element  $\text{cost}[i][j]$  represents the cost of assigning worker  $i$  to task  $j$ . Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function `total_cost(assignment, cost_matrix)` that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function `assignment_problem(cost_matrix)` that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).



```

1 from itertools import permutations
2
3
4 def total_cost(assignment, cost_matrix):
5     cost = 0
6
7     for worker, task in enumerate(assignment):
8         cost += cost_matrix[worker][task]
9
10    return cost
11
12
13 def assignment_problem(cost_matrix):
14     n = len(cost_matrix)
15
16     min_cost = float('inf')
17     best_assignment = None
18
19     for assignment in permutations(range(n)):
20         cost = total_cost(assignment, cost_matrix)
21
22         if cost < min_cost:
23             min_cost = cost
24             best_assignment = assignment
25
26     return best_assignment, min_cost
27
28
29 cost_matrix = [
30     [3, 10, 7],
31     [8, 5, 12],
32     [4, 6, 9]
33 ]
34

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

| Time:       | Memory:  |
|-------------|----------|
| 0.0100 secs | 9.256 Mb |

Your Output

Optimal Assignment:

Worker 1 -> Task 3

Worker 2 -> Task 2

Worker 3 -> Task 1

Total Cost: 16

15. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

- Define a function `total_value(items, values)` that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.
- Define a function `is_feasible(items, weights, capacity)` that takes a list of selected items (represented by their indices), the weight list, and the knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity.

```

1 from itertools import combinations
2
3
4 def total_value(items, values):
5     return sum(values[i] for i in items)
6
7
8 def is_feasible(items, weights, capacity):
9     return sum(weights[i] for i in items) <= capacity
10
11
12 def knapsack(weights, values, capacity):
13     n = len(weights)
14
15     best_value = 0
16     best_items = []
17
18     for r in range(n + 1):
19         for subset in combinations(range(n), r):
20
21             if is_feasible(subset, weights, capacity):
22
23                 value = total_value(subset, values)
24
25                 if value > best_value:
26                     best_value = value
27                     best_items = subset
28
29     return best_items, best_value
30
31
32 weights = [2, 3, 1]
33 values = [4, 5, 3]
34 capacity = 4

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

| Time:       | Memory:  |
|-------------|----------|
| 0.0200 secs | 9.272 Mb |

Your Output

Optimal Selection: [1, 2]

Total Value: 8